



Coláiste na Tríonóide, Baile Átha Cliath
Trinity College Dublin

Ollscoil Átha Cliath | The University of Dublin

Faculty of Engineering, Mathematics and Science
School of Computer Science & Statistics

Senior Freshman
Computer Science and Language
Computer Science and Business

Semester 2 2023

2062 - Natural Language Processing

Thursday 4th May

RDS Simmonscourt

14:00 – 16:00

Dr. Martin Emms

Instructions to Candidates:

There are **4** questions. Answer any **2**

Each question is worth 50 marks

There is a **Supplementary Information** sheet giving some information required for particular questions.

1. Question concerns CFGs and their parsing

(a) let $\mathcal{G}_1$ be the grammar:

initial(vp)	va --> [order]
vp --> L,R	vb --> [parse]
L --> va,np	np --> [it]
R --> [to],vp	np --> [him]
vp --> vb,np	

- i. The input 'order him to parse it' is of category vp according to $\mathcal{G}_1$. Give its analysis tree. [2 marks]
- ii. Give the chart which would be made by the standard CKY parser on this. [8 marks]

(b) Assume the following outline of a CHART class:

```
class CHART {
public:
    Grammar g;
    vector<string> words; // sequence to parse

    /* a member 'chart' such that chart[i][k] stores the categories for
       sub-sequences of the input with start = i and length = k */

    bool accept(vector<string> input);
    void initialize(); // set chart dimensions then put in length 1 entries
    void do_cell(int i, int k); // fill in all entries at chart[i][k]
    bool is_there(int i, int k, Category c); // check for c at chart[i][k]
    void add_edge(int i, int k, Category c); // add c to chart[i][k] if not there
};
```

Assume `accept(..)` calls `initialize()` to add all length 1 entries and then `do_cell(i,k)` is repeatedly called in such a way that, for all start positions i and lengths k , every category C goes into `chart[i][k]` which is implied by rules applied to existing entries for shorter lengths k' at appropriate positions.

Assuming `is_there(..)` and `add_edge(..)` are already implemented, give an implementation of `do_cell(..)`. [15 marks]

(question continues next page)

- (c) Suppose $\mathcal{G}_2$ is the grammar that derives from $\mathcal{G}_1$ by replacing the 3 rules (α) below, with the single rule (β) below.

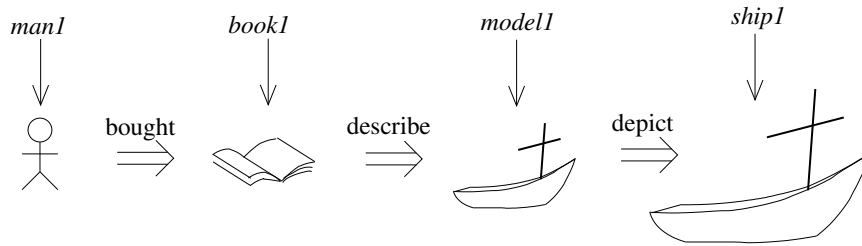
(α) $vp \rightarrow L, R$ $L \rightarrow va, np$ $R \rightarrow [to], vp$ (β) $vp \rightarrow va, np, [to], vp$

- i. State the aspect of $\mathcal{G}_2$ which makes it not suited to the standard CKY parser [2 marks]
 - ii. There is an adaptation of the CKY algorithm using '*partial constituents*', with categories of the form $C/d_i \dots d_n$, which makes it suited to a grammar such as $\mathcal{G}_2$. Assuming this adaptation give the chart that will be produced on the input '*order him to parse it*'. [12 marks]
- (d) Now let $\mathcal{G}_3$ be the extension of $\mathcal{G}_2$ with the further rules

$np \rightarrow det, n$
 $det \rightarrow [a]$
 $n \rightarrow n, n$
 $n \rightarrow [child]$
 $n \rightarrow [grammar]$
 $n \rightarrow [expert]$

- i. give the two np trees for '*a child grammar expert*'. [2 marks]
- ii. Let **acge** abbreviate '*a child grammar expert*'. The sentence '*order **acge** to order **acge** to parse it*' has 4 analyses. How many different analyses will '*(order **acge** to)ⁿ parse it*' have? [3 marks]
- iii. In both variants of the CKY algorithm, $chart[i][k]$ stores all the possible categories of sequences starting at i and of length k , and the algorithm has better than exponential running time. Briefly explain why this could not be so if the cells stored not all *categories* but instead all the *trees* for those categories. [6 marks]

2. Question concerns Centre-embedding and Slash-grammars



- (a) Referring to the above picture, *ship1* might be identified via its relation to *model1* as in (i) below, and *model1* via its relation to *book1* as in (ii) below.

(i) *the ship that model1 depicted* (ii) *the model that book1 described*

When (ii)'s description of *model1* is substituted into (i) the result is a noun-phrase with an object relative clause which contains a further such noun-phrase. Give this noun-phrase [2 marks]

- (b) The process could be continued, for example replacing *book1* with *the book that man1 bought*. Let *NPCE* stand for the language obtained by repeatedly doing this, with contents defined as follows:

$$NPCE = \{(\text{the } N \text{ that})^n PN (TV)^n \mid n \geq 1\} \text{ where}$$

$$N \in \{ship, model, book, man\} \quad TV \in \{depicted, described, bought\}$$

$$PN = \{ship1, model1, book1, man1\}$$

- i. Give a CFG which also defines the set *NPCE*. [6 marks]
 - ii. *NPCE* is very analogous to the language $\mathcal{P} = \{a^n b^n \mid n \geq 1\}$. Outline the proof using the Pumping Lemma (see **Supplementary Info**) which shows that $\mathcal{P}$ *could not* be defined by an FSA [8 marks]
- (c) Suppose the language *NPRE* is defined as follows:

$$NPRE = \{(\text{the } N \text{ that } TV)^n PN \mid n \geq 1\} \text{ where}$$

$$N, TV, PN \text{ are as for } NPCE$$

- i. Give a CFG which also defines the set *NPRE*. [6 marks]
- ii. Give also an FSA which defines *NPRE*. [3 marks]

(question continues next page)

- (d) The relative clauses in *NPCE* and *NPRE* are of very particular forms, special cases of the correct generalisation, which is that what follows the relative pronoun should be

any sentence from which some noun-phrase has been removed **(Rel Gen)**

'slash grammars' capture this generalisation by adding rules for categories like *s/np* and its kin so that *s/np* categorises the strings fitting the generalisation **Rel Gen** above.

Let $\mathcal{G}$ be the following CFG

<i>s</i> --> <i>np, vp</i>	<i>det</i> --> [<i>the</i>]	<i>np</i> --> [<i>man1</i>]	<i>tv</i> --> [<i>bought</i>]
<i>np</i> --> <i>det, n</i>	<i>n</i> --> [<i>man</i>]	<i>np</i> --> [<i>book1</i>]	<i>tv</i> --> [<i>described</i>]
<i>np</i> --> <i>pn</i>	<i>n</i> --> [<i>book</i>]	<i>np</i> --> [<i>model1</i>]	<i>tv</i> --> [<i>depicted</i>]
<i>vp</i> --> <i>tv, np</i>	<i>n</i> --> [<i>model</i>]	<i>np</i> --> [<i>ship1</i>]	<i>dtv</i> --> [<i>showed</i>]
<i>vp</i> --> <i>dtv, np, [to], np</i>	<i>n</i> --> [<i>ship</i>]	<i>np</i> --> [<i>Noel</i>]	
		<i>np</i> --> [<i>Matt</i>]	

Extend it to a larger CFG allowing for nps with relative clauses, based on the **Rel Gen** generalisation, by adding suitable rules involving *s/np* (and others of the form *X/np*).

Ensure as you do so that you achieve coverage of the following, drawing the trees according to your grammar to verify this

the ship that model1 depicted

the model that depicted ship1

the man that Noel showed the book to [17 marks]

- (e) $\mathcal{G}$ lacks verbs which allow an *s* to be a sub-constituent of a larger *s*, as in

Noel believed *model1 depicted ship1*

Matt said *Noel believed model1 depicted ship1*

Add rules to allow these and the further nps with relative clauses such as:

the ship that Noel believed model1 depicted

the ship that Matt said Noel believed model1 depicted [5 marks]

- (f) Draw the analysis tree according to your grammar of

the ship that Noel believed model1 depicted [3 marks]

3. Question concerns Feature structures in grammars

- (a) The idea that one feature structure can *specialise* into another is a key one, and in terms of it the notion of *unification* of two feature structures can be defined. Give such a definition. [4 marks]
- (b) For the feature structures below you should consider whether they *unify*, giving the unified feature structure when they do. They are presented as *typed* feature structures. Consider each first as if *untyped*, and then again noting the types. Assume the hierarchy $\mathcal{H}$ given in the Supplementary Info. [10 marks]

$$\begin{array}{ll}
 \text{(i)} \begin{bmatrix} agr \\ num \quad pl \\ pers \quad third \end{bmatrix} & \begin{bmatrix} nflex \\ cse \quad acc \end{bmatrix} & \text{(ii)} \begin{bmatrix} vnonfin \\ vform \quad base \end{bmatrix} & \begin{bmatrix} vnonfin \\ vform \quad prog \end{bmatrix} \\
 \text{(iii)} \begin{bmatrix} vnonfin \\ vform \quad prog \end{bmatrix} & \begin{bmatrix} vfin \\ pers \quad 1 \\ num \quad sing \\ tns \quad pres \end{bmatrix}
 \end{array}$$

- (c) In English there are some *number* requirements concerning the determiner word at the beginning of a noun-phrase and the noun at the end of a noun-phrase, as illustrated by

most big furry cats
most big furry cat *
a big furry cats *
a big furry cat

Let $\mathcal{L}$ be defined as the following fragment of English illustrating this

every expression in $\mathcal{L}$
begins with a member of $\{\text{most}, a\}$
is followed by $n \geq 0$ members of $\{\text{big}, \text{furry}\}$
and ends with
a member of $\{\text{cats}, \text{rabbits}\}$ if the initial word was most
a member of $\{\text{cat}, \text{rabbit}\}$ if the initial word was a

CFGs enhanced with *feature structures* allow such phenomena to be described concisely. Give such a grammar $\mathcal{G}_1$ for $\mathcal{L}$. Don't worry about the particular notation in which the rules are expressed, so long as it clearly pairs category names with feature structures. [16 marks]

(question continues next page)

- (d) In using a featurized grammar, if its rules do not mention a particular feature at all in association with a particular word, does that exclude that word's appearance in position's where a particular value for that feature is required? [4 marks]

- (e) Unlike *most* and *a*, the word *the* does not place requirements on the number of the final noun:

the big furry cats

the big furry cat

Extend your grammar to $\mathcal{G}_2$ to deal with this also, and say in a few words how specialisation and unification permit the above to be parsed when $\mathcal{G}_2$ is used.

[8 marks]

- (f) In the course both untyped and typed feature structures were discussed. Briefly discuss a possible advantage to the use of typed feature structures. [8 marks]

4. Question concerns computing a semantic value

Note this question makes reference to material in the **Supplementary Info** sheet.

- (a) Provide code implementing `Rel reduce(Rel r, int which, Thing v)`, using the helper function `Tuple erase(Tuple t, int i)` [12 marks]

(to illustrate, a call on the `likes` relation in the Supplementary Info is shown below:

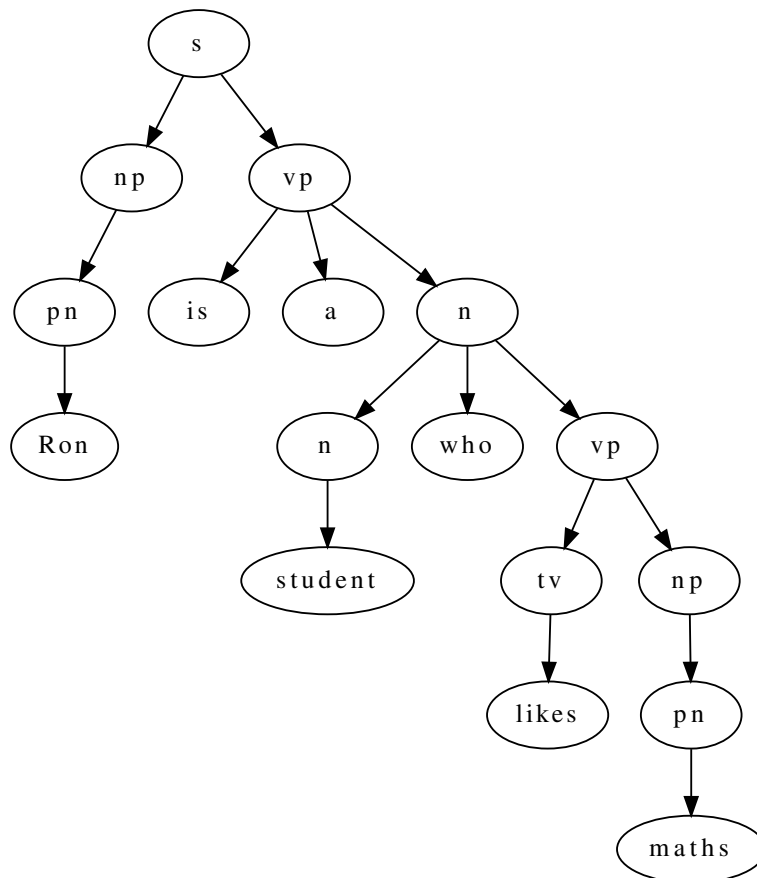
`Rel p(1); p = reduce(likes, 1, "maths");` $\Rightarrow$

Martin
Hermione
Ron

p

)

- (b) Give an implementation of a function `Rel meet(Rel p, Rel q)` such that given arity-1 Rel objects *p* and *q* it returns the *intersection* of these. [6 marks]
- (c) Consider computing a semantic value for the tree

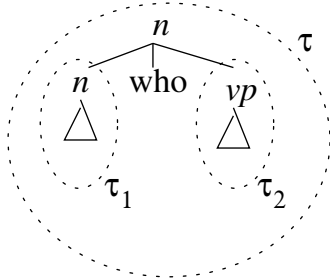


(question continues next page)

Suppose the case of `Rel compute_vp_value(Tree *t)` suited to the upper `vp` node equates the `vp`'s value with that of its `n` part. Concerning other parts of the tree, complete the parts enclosed in `/* .. */` in the partial semantic evaluation rules below:

(i) suited to the uppermost `n` node

[10 marks]



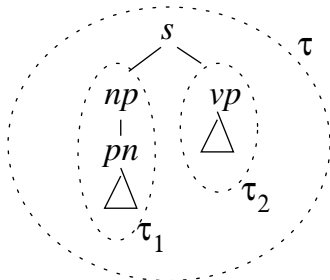
```

Rel compute_n_value(Tree *tau) {
    if( $\tau, \tau_1, \tau_2$  are as shown to the left) {
        Rel r(1);
        /* code using compute_vp_value(..)
           compute_n_value(..) */
        return r;
    }
    else { poss other cases}
}

```

(ii) suited to the `s` node

[10 marks]



```

bool compute_s_value(Tree *tau) {
    bool v;
    if( $\tau, \tau_1, \tau_2$  are as shown to the left) {
        /* code using compute_np_value(..)
           compute_vp_value(..) */
        return v;
    }
    else { poss other cases}
}

```

(iii) Consider

(α) Ron is a student who likes maths

(β) Ron is a student

(γ) Ron likes maths

No matter what the settings of the relations, the semantic evaluation functions are such that it will never be the case that (α) gets value true whilst either of (β) or (γ) gets value false. Show that this is the case.

[12 marks]